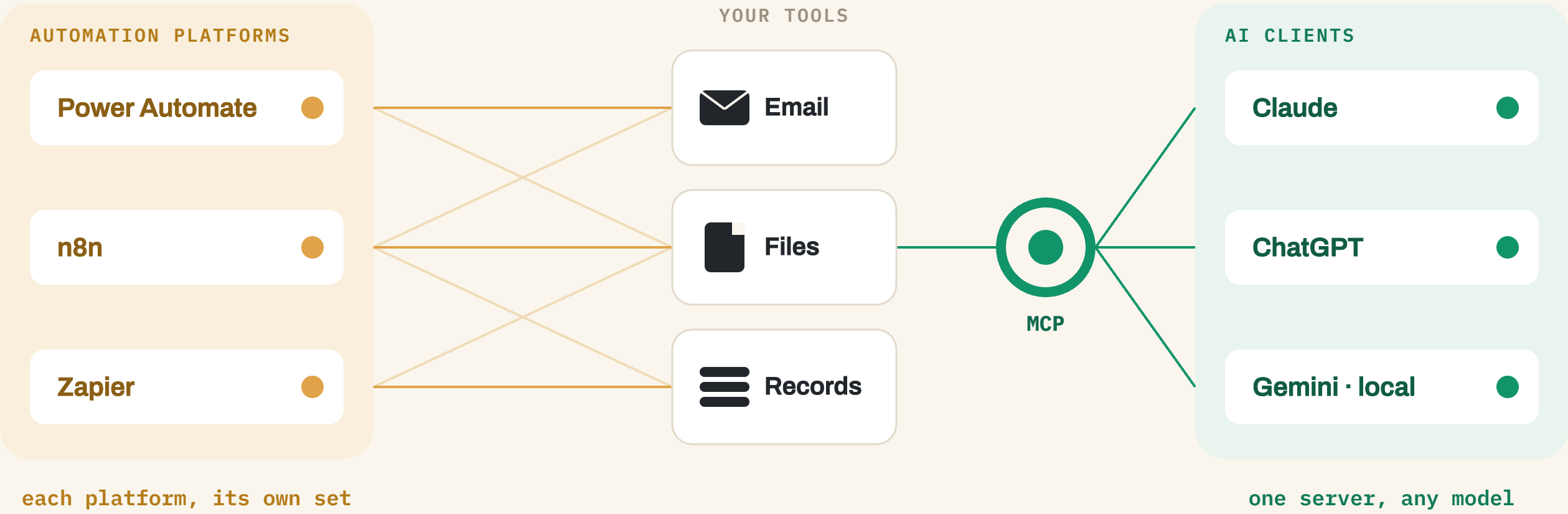


MCP vs Connectors

Same job. Opposite philosophy.



what each one is • who fills the values • who builds it • when to use which

Two answers to the same question

Software can't do anything useful until it can reach your tools. Automation platforms and AI models each solved that problem — separately.

CONNECTOR

The platform's answer

Built by the vendor, for that platform.

Gmail

Drive

ERP

AUTOMATION PLATFORM

a different plug for every app

- A pre-built adapter for one app — Gmail, SharePoint, Slack.
- Ships inside the platform. You pick it from a list.
- Every platform rebuilds its own version of the same thing.

MCP

The AI's answer

An open protocol, not a product.

Claude

GPT

Gemini

ONE STANDARD PORT

same plug, any model

- A small server that describes what a tool can do.
- Any model that speaks MCP can use it — no rewrite.
- Written once, reused across every AI app you own.

Different words. Same anatomy.

Strip the branding and both are the same object: a list of things a tool can do, plus the inputs each one needs.

CONNECTOR

A catalogue of actions

Written by the platform vendor.

Gmail connector

Send an email

ACTION

Get emails

TRIGGER

Move to folder

ACTION

shipped + versioned by the vendor

- Each action has fixed inputs — To, Subject, Body, Attachment.
- Triggers, actions and auth all come bundled together.
- You meet it as a step inside the flow designer.

MCP

A catalogue of tools

Written by whoever runs the server.

mail-server

send_email

TOOL

list_messages

TOOL

move_message

TOOL

you can read – or write – the source

- Each tool ships a described input schema, in plain language.
- The model reads that description at runtime, not at build time.
- The AI meets it as something it may choose to call.

Same job: send an email

One task, two completely different chains of custody. The question that matters is simple: who does the thinking, and when?

CONNECTOR

You build the chain

Every link — before it ever runs.

- 1 Pick the connector**
You choose "Office 365 Outlook" from the platform's own list of available apps.
- 2 Pick the action**
"Send an email (V2)" — one option from a fixed, published set. Nothing else exists.
- 3 Fill in every field**
To, Subject, Body — each one mapped by hand from an earlier step in the flow.
- 4 It runs, exactly so**
Same fields, same order, every single time. Change the shape and you edit the flow.

MCP

The model builds it

At the moment it runs.

- 1 You state the intent**
"Email the team the summary from that thread." That is the whole instruction.
- 2 It reads what's available**
The server tells it send_email exists and which arguments that tool expects.
- 3 It writes the arguments**
Recipients, subject and body — composed from the context, not from your mapping.
- 4 It calls, checks, retries**
Wrong address? It reads the error it gets back and tries again on its own.

Someone has to write the values

This is the real dividing line. Both need arguments. The difference is whether a person types them in advance — or a model works them out on the spot.

CONNECTOR

You type them in

Design time, by hand.

Send an email

TO

`outputs('Get_user')['mail']`

SUBJECT

`Weekly summary`

MESSAGE BODY

`body('Create_HTML_table')`

- Every field mapped, every branch anticipated in advance.
- An edge case you didn't map is a run that fails.
- In exchange: it does the same thing, perfectly, forever.

MCP

The model works them out

Run time, from context.

"send the team this week's summary"



ARGUMENTS THE MODEL WROTE

`to: team@company.com`

`subject: Weekly summary`

`body: written from the thread`

- It infers the values from what you said and what it just read.
- A new edge case? It adapts without you editing anything.
- In exchange: less control — and it can still get it wrong.

One you wait for. One you can write.

If the connector you need doesn't exist, you file a request and wait. If the MCP server you need doesn't exist, you can write it this afternoon.

CONNECTOR

You wait for the vendor

Their catalogue, their roadmap.

Connector catalogue

Outlook

Teams

SQL

SharePoint

Drive

your app?

not in the list? file a request.

- The provider owns it. New capability means waiting for a new release.
- Auth, rate limits and quirks arrive exactly as given.
- Upside: it is maintained, documented and supported for you.

MCP

You can build your own

And host it wherever you like.

Your own server

```
@tool
def close_ticket(id):
    return crm.close(id)
```

your laptop

your server

your VPC

no queue, no roadmap, no ticket.

- Anything with an API can become a tool you own.
- Self-host it and the data never leaves your own network.
- Downside: you own the auth, the limits and every fix.

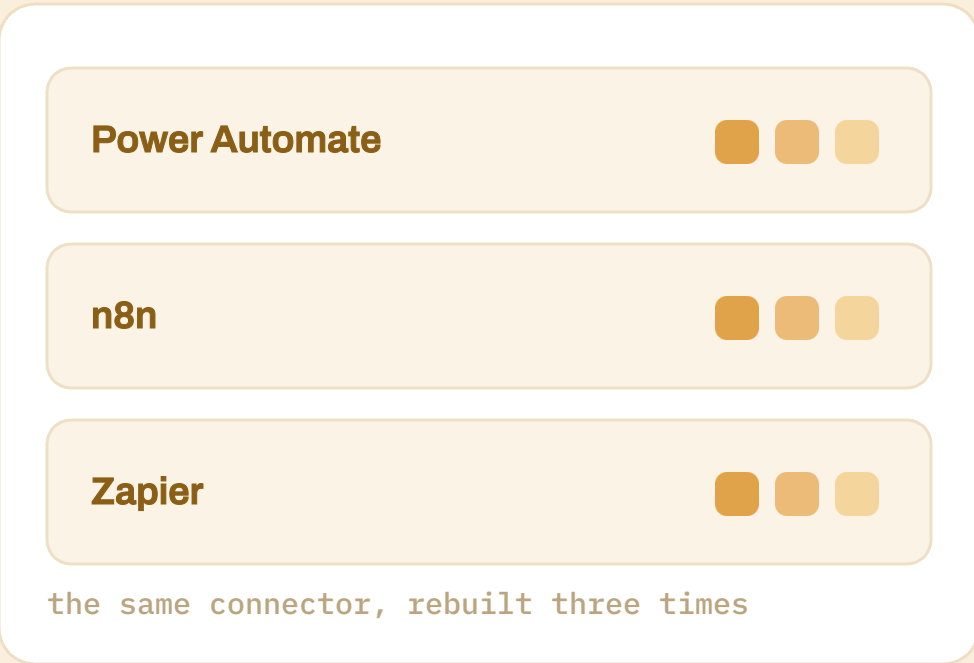
Locked in one place, or loose everywhere

A connector belongs to its platform. An MCP server belongs to nobody in particular — which is exactly why it travels.

CONNECTOR

One platform at a time

The work does not transfer.

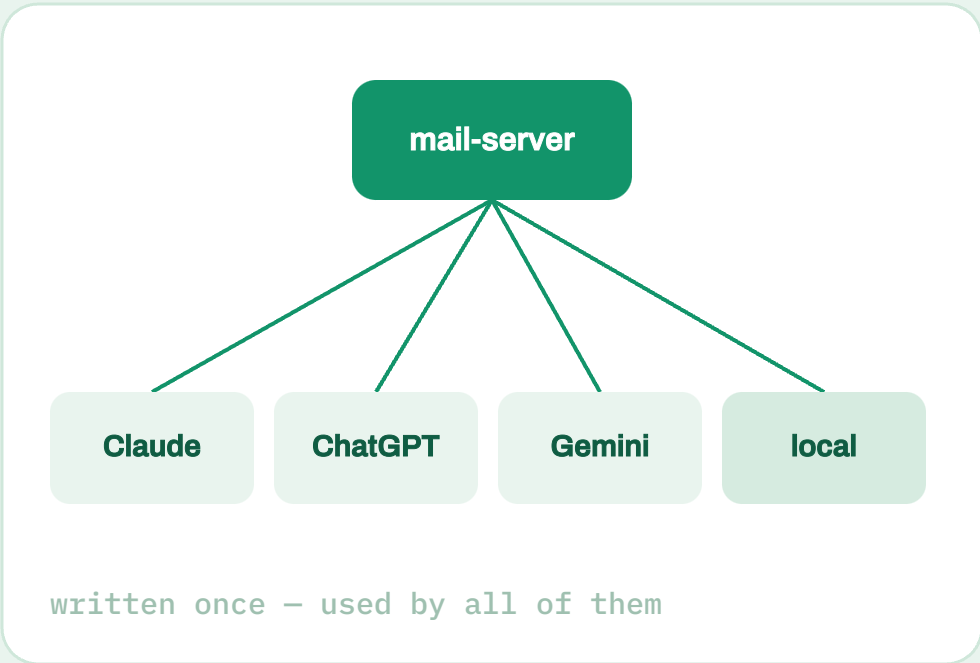


- Built it in Power Automate? It does not move to n8n.
- Each platform has its own set, and its own gaps.
- Switch platform and you rebuild the plumbing from scratch.

MCP

Every client at once

One server, many models.



- Claude, ChatGPT, Gemini — or a model running on your own hardware.
- The same server answers your IDE, your terminal and your desktop app.
- A new model lands next year? It simply connects. Nothing to rewrite.

Neither one wins

They are not competitors. They are opposite tools built for opposite jobs — and the cost profile tells you, every time, which one you are actually holding.

CONNECTOR

Cheap, fast, literal

SPEED PER STEP

Milliseconds

It runs on rails. No thinking required.

COST

Near zero

Predictable per run, at any volume.

WHEN THINGS CHANGE

It breaks

An input you didn't map is a failed run.

WHAT IT IS FOR

Repetitive work

Same shape, high volume, 24/7.

MCP

Slow, costly, adaptive

SPEED PER STEP

Seconds

It has to reason before it acts.

COST

Real money

Tokens on every single call it makes.

WHEN THINGS CHANGE

It adapts

It reads the situation and adjusts.

WHAT IT IS FOR

Novel work

One-off, messy, needs judgement.

Which one do you reach for?

A simple test. Read both columns and notice which one your next task falls into. Most weeks, honestly, it will be both.

CONNECTOR

Reach for a connector

when the shape is already known.

- ✓ The steps are the same every single time you do it.
- ✓ It has to run at 3am, on schedule, without you watching.
- ✓ Volume is high and the cost per run actually matters.
- ✓ You can describe the whole thing before it ever runs.

Define it once → it runs forever.

MCP

Reach for MCP

when the shape is not.

- ✓ Every instance of the task looks a little bit different.
- ✓ The input is messy language, not clean fields.
- ✓ You would need forty branches to cover every case.
- ✓ You want to try the idea before you commit to building it.

Describe it once → it works it out.

Learn both. They answer different questions.

A connector is a promise you make in advance.

You define it down to the last field. It keeps its word, forever, for almost nothing.

An MCP tool is a capability you hand over.

You describe what it can do. The model decides when, and works out the rest.

Almost every real job needs a bit of each.

The skill is knowing which half of the problem you are holding at the time.

Build one small MCP server this month. It is the cheapest way
to feel the difference for yourself.